Dossier de Registro: padic-grn-core

Núcleo de análisis p-ádico para sistemas dinámicos discretos

Jesús Rogelio Pérez-Buendía Víctor Nopal-Coello Yuriria Cortés-Poza

2026-04-25

Abstract

`padic-grn-core` es el núcleo matemático y computacional para calcular métricas p-ádicas (`mu_E`, `mu_A`, `mu_I`) en redes de regulación genética y, con adaptaciones controladas, en sistemas dinámicos discretos finitos más generales. Este dossier reúne documentación funcional, código fuente, pruebas y evidencia de validación reproducible para fines de registro de software.

Contents

1 Introducción **3**

2 Datos generales **3**
2.1 Correos electrónicos oficiales de autores . . . 3
2.2 Financiamiento . . . 3

3 Autoría oficial **3**

4 Documentación funcional completa **4**
4.1 README . . . 4
4.2 Manual de usuario . . . 5
4.3 Manual técnico . . . 6
4.4 Configuración de paquete . . . 8
4.5 Cita académica . . . 9
4.6 Licencia . . . 9
4.7 Configuración de pruebas . . . 9

5 Código fuente completo **10**
5.1 Inicialización de paquete . . . 10
5.2 CLI . . . 10
5.3 Adaptadores . . . 12
5.4 Script de validación . . . 13

6 Pruebas y ejemplos completos **16**
6.1 Prueba smoke . . . 16
6.2 Pruebas de adaptadores . . . 16
6.3 Ejemplo de transiciones p=2 . . . 16
6.4 Ejemplo de transiciones p=3 . . . 17
6.5 Ejemplo de orderings . . . 17

7 Evidencia textual de validación **17**
7.1 Log de ejecución de pruebas 17
7.2 Salida JSON toy p=2 17
7.3 Salida JSON toy p=3 18
7.4 Salida CSV batch 18
7.5 Salida A. thaliana 18
7.6 Salida C. elegans 20
7.7 Salida Drosophila 21

1 Introducción

Este software centraliza el cálculo de perfiles p-ádicos de expansión, contracción e indiferencia sobre dinámicas discretas deterministas. Su diseño prioriza trazabilidad científica, reproducibilidad y reutilización por terceros: incluye interfaz de línea de comandos, adaptadores de entrada, pruebas automatizadas y ejemplos mínimos verificables.

El objetivo práctico de `padic-grn-core` es ofrecer una base confiable sobre la cual puedan construirse módulos de búsqueda (`padic-grn-search`) y de visualización (`padic-grn-viz`), manteniendo consistencia matemática y operativa en todo el ecosistema.

2 Datos generales

- Nombre: `padic-grn-core`
- Versión: `0.1.0`
- Lenguaje: Python 3.10+
- Tipo: software de cálculo de métricas p-ádicas para sistemas discretos finitos
- DOI: pendiente de asignación institucional

2.1 Correos electrónicos oficiales de autores

- Jesús Rogelio Pérez-Buendía: `rogelio.perez@cimat.mx`
- Víctor Nopal-Coello: `victor_nopal@uaeh.edu.mx`
- Yuriria Cortés-Poza: `yuriria.cortes@iimas.unam.mx`

2.2 Financiamiento

- Modelos matemáticos y computacionales no convencionales para el estudio y análisis de problemas relevantes en Biología, CF 2019/217367.

3 Autoría oficial

```
# Autoría, ORCID y DOI oficial (softwares GRN)

Fecha: 2026-04-25

Este documento fija la información oficial de autoría para:

- `padic-grn-core`
- `padic-grn-search`
- `padic-grn-viz`

## Orden oficial de autoría

1. **Jesús Rogelio Pérez-Buendía**
   SECIHTI - Centro de Investigación en Matemáticas (CIMAT), Unidad Mérida (Mérida,
   ↪  Yucatán, México).
```

```
   ORCID: `0000-0002-7739-4779`

2. **Víctor Nopal-Coello**
   Universidad Autónoma del Estado de Hidalgo (UAEH) (Pachuca, Hidalgo, México).
   ORCID: `0000-0003-2608-3636`

3. **Yuriria Cortés-Poza**
   IIMAS, UNAM - Unidad Académica Yucatán (Mérida, Yucatán, México).
   ORCID: `0000-0003-1856-9206`

## Fuente de verificación (artículo y acuerdo interno)

- `20250115-p-adic-GRN-rewrite/articulo-final/parte-II.2/main-PII-2.tex`
  - autores y ORCID en bloque de autores
  - afiliaciones en bloque `\affil[...]`
- Acuerdo explícito de orden de autoría definido por el equipo para el paquete de
↪  software.

## DOI por software

- `padic-grn-core`: **pendiente de asignación** (Zenodo/release).
- `padic-grn-search`: **pendiente de asignación** (Zenodo/release).
- `padic-grn-viz`: **pendiente de asignación** (Zenodo/release).

Al asignarse, registrar aquí:

- DOI,
- versión/tag correspondiente,
- URL de depósito.

## Financiamiento (funding)

Modelos matemáticos y computacionales no convencionales para el estudio y análisis de
↪  problemas relevantes en Biología, CF 2019/217367.
```

4 Documentación funcional completa

4.1 README

```
# padic-grn-core: Núcleo p-ádico para sistemas dinámicos discretos

## Resumen

Paquete base para calcular métricas p-ádicas (`mu_E`, `mu_A`, `mu_I`) en sistemas
↪  dinámicos discretos finitos.

## Introducción

`padic-grn-core` concentra el cálculo matemático central de la suite. Está orientado a
↪  redes de regulación genética multivaluadas y, mediante adaptadores, puede emplearse
↪  también en sistemas discretos generales.

## Comandos
```

- `pgcore-score-ordering`
- `pgcore-batch-scores`

Uso rápido

```bash
pgcore-score-ordering transitions.csv 13 2 "12 4 3 8 1 5 7 2 10 11 9 6 0" --output-json
↪  scores.json
```

`pgcore-batch-scores` espera un CSV con columna `ordering`.

Alcance objetivo

- Soporte oficial: GRN deterministas multivaluadas.
- Soporte experimental: sistemas discretos generales via adaptadores.

Autoría oficial

1. Jesús Rogelio Pérez-Buendía (SECIHTI-CIMAT, Unidad Mérida) - ORCID
↪ `0000-0002-7739-4779`
2. Víctor Nopal-Coello (Universidad Autónoma del Estado de Hidalgo, UAEH) - ORCID
↪ `0000-0003-2608-3636`
3. Yuriria Cortés-Poza (IIMAS, UNAM - Unidad Académica Yucatán) - ORCID
↪ `0000-0003-1856-9206`

DOI

- DOI del software: pendiente de asignacion institucional.

Financiamiento

Modelos matemáticos y computacionales no convencionales para el estudio y análisis de
↪ problemas relevantes en Biología, CF 2019/217367.

4.2 Manual de usuario

Manual de usuario - padic-grn-core

1. Requisitos

- Python 3.10+
- Archivo CSV de transiciones

2. Instalación local (modo desarrollo)

```bash
cd software-suite/padic-grn-core
pip install -e .
```

3. Uso básico: un ordenamiento

```bash
```

```
pgcore-score-ordering transitions.csv 13 2 "12 4 3 8 1 5 7 2 10 11 9 6 0" --output-json
↪  scores.json
```

Resultado:

- genera `scores.json` con `mu_E`, `mu_A`, `mu_I`.

4. Uso por lote (batch)

`orderings.csv` debe tener una columna `ordering`.

```bash
pgcore-batch-scores transitions.csv 13 2 orderings.csv --output-csv batch_scores.csv
```

Resultado:

- `batch_scores.csv` con columnas:
  - `ordering`
  - `mu_E`
  - `mu_A`
  - `mu_I`

5. Buenas prácticas

- usar rutas relativas limpias y evitar espacios en nombres de archivos de trabajo.
- guardar siempre los parámetros de corrida junto al resultado.
- versionar los archivos de entrada críticos.

6. Solución de problemas

- Error "ordering inválido":
  - verifica que contenga todos los índices `0..N-1` sin repetición.
- Error de CSV:
  - verifica columnas `config_from,config_to`.
- Error de módulo legado no encontrado:
  - ejecutar desde el workspace donde existe el árbol fuente oficial.

4.3 Manual técnico

```
# Manual técnico - padic-grn-core

## 1. Propósito científico

`padic-grn-core` implementa el núcleo matemático para cuantificar estabilidad jerárquica
↪  en sistemas dinámicos discretos finitos mediante métricas p-ádicas:

- `mu_E` (expansión),
- `mu_A` (contracción),
- `mu_I` (isometría/indiferencia),
- perfiles por nivel `E(n), A(n), I(n)`.

Este módulo es la base de `padic-grn-search` y `padic-grn-viz`.
```

```
## 2. Modelo matemático formal

Sea `f: C -> C` un sistema discreto determinista sobre `|C| = p^N`.

1. Se fija un ordenamiento `pi` de variables.
2. Se identifica cada estado con su expansión base `p`.
3. Para cada nivel `n = 1, ..., N-1`, el espacio se particiona en cilindros/bolas
↪  `B_{n,m}`.
4. Para cada bola, se calcula `M_{n,m}` = primer nivel de discrepancia en imágenes bajo
↪  `f`.
5. Se clasifica:
   - `E` si `M_{n,m} < n`,
   - `I` si `M_{n,m} = n`,
   - `A` si `M_{n,m} > n`.
6. Con peso `w_n = p^(N-n)`:
   - `mu_E = sum_n E(n) * w_n`
   - `mu_I = sum_n I(n) * w_n`
   - `mu_A = sum_n A(n) * w_n`

Invariante de consistencia por nivel:

- `E(n) + I(n) + A(n) = p^n`.

## 3. Arquitectura de implementación

- `src/padic_grn_core/cli.py`
  - `pgcore-score-ordering`
  - `pgcore-batch-scores`
- `src/padic_grn_core/adapters.py`
  - `from_grn_table(...)` (estable)
  - `from_discrete_system(...)` (beta)

Motor actual:

- En esta fase, la CLI se conecta al módulo legado validado:
  - `20250115-p-adic-GRN-rewrite/articulo-final/supplementary/code/core/compute_mu_gener
  ↪  alized.py`

## 4. Contrato de datos

Entrada mínima:

- `N` (entero positivo),
- `p` (entero >= 2),
- `transition_table.csv` con columnas:
  - `config_from`
  - `config_to`,
- `ordering` como permutación de `0..N-1`.

Salida estándar (`scores.json`):

- metadatos (`software`, `version`, `N`, `p`, `ordering`),
- `scores.mu_E`, `scores.mu_A`, `scores.mu_I`,
- perfiles por nivel.
```

```
## 5. Complejidad y límites prácticos

- Cálculo de una permutación: depende de `sum_{n=1}^{N-1} p^n` y del costo por bola.
- Para `N` grande, el cuello de botella está en evaluación masiva de bolas.
- El módulo soporta paralelismo en backend legado; la capa empaquetada lo expone
↪  indirectamente.

## 6. Validación y trazabilidad

Validaciones obligatorias:

- ordering válido (permutación),
- CSV con cabeceras correctas,
- integridad de tabla de transición.

Trazabilidad:

- toda corrida debe guardar: parámetros, versión, ruta de datos, hash de entrada
↪  (pendiente de incluir en CLI v0.2).

## 7. Roadmap técnico inmediato

1. Migrar internals del módulo legado a `src/padic_grn_core/engine.py`.
2. Añadir hash de entrada y checksum de salida.
3. Añadir pruebas deterministas para `p=2` y `p=3` en CI.
4. Añadir API python estable:
   - `compute_scores(...)`,
   - `validate_transition_table(...)`.
```

4.4 Configuración de paquete

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padic-grn-core"
version = "0.1.0"
description = "Core general para metricas p-adicas en sistemas discretos."
readme = "README.md"
requires-python = ">=3.10"
license = {text = "Proprietary - Pending institutional definition"}
authors = [
  {name = "Jesús Rogelio Pérez-Buendía", email = "rogelio.perez@cimat.mx"},
  {name = "Víctor Nopal-Coello", email = "victor_nopal@uaeh.edu.mx"},
  {name = "Yuriria Cortés-Poza", email = "yuriria.cortes@iimas.unam.mx"}
]
dependencies = []

[project.scripts]
pgcore-score-ordering = "padic_grn_core.cli:main_score_ordering"
pgcore-batch-scores = "padic_grn_core.cli:main_batch_scores"

[tool.setuptools]
package-dir = {"" = "src"}
```

```
[tool.setuptools.packages.find]
where = ["src"]
```

4.5 Cita académica

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
title: "padic-grn-core"
version: "0.1.0"
type: software
authors:
  - family-names: "Pérez-Buendía"
    given-names: "Jesús Rogelio"
    affiliation: "SECIHTI - Centro de Investigación en Matemáticas (CIMAT), Unidad
    ↪  Mérida"
    orcid: "https://orcid.org/0000-0002-7739-4779"
  - family-names: "Nopal-Coello"
    given-names: "Víctor"
    affiliation: "Universidad Autónoma del Estado de Hidalgo (UAEH)"
    orcid: "https://orcid.org/0000-0003-2608-3636"
  - family-names: "Cortés-Poza"
    given-names: "Yuriria"
    affiliation: "Instituto de Investigaciones en Matemáticas Aplicadas y en Sistemas
    ↪  (IIMAS), UNAM -- Unidad Académica Yucatán"
    orcid: "https://orcid.org/0000-0003-1856-9206"
abstract: "Core general para cálculo de métricas p-ádicas (mu_E, mu_A, mu_I) y análisis
↪  base en sistemas dinámicos discretos."
license: "Proprietary - Pending institutional definition"
repository-code: "PENDING_REPOSITORY_URL"
doi: "10.0000/pending-doi"
```

4.6 Licencia

```
Proprietary License Notice
Copyright (c) 2026
Jesús Rogelio Pérez-Buendía, Víctor Nopal-Coello, Yuriria Cortés-Poza

This software is currently released for academic evaluation and registration
workflow preparation. Redistribution, sublicensing, and commercial use are not
authorized without explicit written permission from the rights holders and
institutional approval.

Final license terms may be updated in the public release.
```

4.7 Configuración de pruebas

```
[pytest]
pythonpath = src
```

5 Código fuente completo

5.1 Inicialización de paquete

```
"""padic-grn-core package."""

__all__ = ["__version__"]
__version__ = "0.1.0"
```

5.2 CLI

```
"""CLI funcional para padic-grn-core."""

from __future__ import annotations

import argparse
import csv
import importlib.util
import json
import sys
from pathlib import Path
from typing import Any


def _workspace_root() -> Path:
    current = Path(__file__).resolve()
    for parent in current.parents:
        if (parent / "20250115-p-adic-GRN-rewrite").exists():
            return parent
    raise RuntimeError("No se encontró la raíz del workspace.")


def _load_core_module() -> Any:
    root = _workspace_root()
    module_path = (
        root
        / "20250115-p-adic-GRN-rewrite"
        / "articulo-final"
        / "supplementary"
        / "code"
        / "core"
        / "compute_mu_generalized.py"
    )
    if not module_path.exists():
        raise FileNotFoundError(f"No existe el módulo core: {module_path}")
    spec = importlib.util.spec_from_file_location("legacy_compute_mu_generalized",
    ↪  module_path)
    if spec is None or spec.loader is None:
        raise RuntimeError("No se pudo cargar el módulo core legado.")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def _parse_ordering(ordering_text: str, n: int) -> list[int]:
```

```
    order = [int(x) for x in ordering_text.replace(",", " ").split()]
    if sorted(order) != list(range(n)):
        raise ValueError(f"ordering inválido; debe ser permutación de 0..{n-1}")
    return order


def _write_json(output_path: Path, payload: dict[str, Any]) -> None:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, ensure_ascii=False)


def main_score_ordering() -> None:
    parser = argparse.ArgumentParser(
        description="Calcula mu_E, mu_A, mu_I para un ordering dado."
    )
    parser.add_argument("transitions_csv", help="CSV con columnas config_from,config_to")
    parser.add_argument("N", type=int, help="Número de variables")
    parser.add_argument("p", type=int, help="Base discreta")
    parser.add_argument("ordering", help='Permutación, ej. "0 1 2 3" o "0,1,2,3"')
    parser.add_argument(
        "--output-json",
        default="scores.json",
        help="Archivo de salida JSON (default: scores.json)",
    )
    args = parser.parse_args()

    try:
        module = _load_core_module()
        table = module.read_transition_table_csv(args.transitions_csv, args.p)
        ordering = _parse_ordering(args.ordering, args.N)
        scores = module.compute_scores(args.p, args.N, ordering, table,
        ↪  return_profiles=True)
        payload = {
            "software": "padic-grn-core",
            "version": "0.1.0",
            "N": args.N,
            "p": args.p,
            "ordering": ordering,
            "scores": scores,
        }
        out = Path(args.output_json).resolve()
        _write_json(out, payload)
        print(f"OK: resultados guardados en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgcore-score-ordering: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_batch_scores() -> None:
    parser = argparse.ArgumentParser(
        description="Calcula scores para una lista de orderings (batch)."
    )
    parser.add_argument("transitions_csv", help="CSV con transiciones")
    parser.add_argument("N", type=int, help="Número de variables")
```

```
    parser.add_argument("p", type=int, help="Base discreta")
    parser.add_argument(
        "orderings_csv",
        help="CSV con columna ordering (string con enteros separados por espacio/coma)",
    )
    parser.add_argument(
        "--output-csv",
        default="batch_scores.csv",
        help="Archivo de salida CSV (default: batch_scores.csv)",
    )
    args = parser.parse_args()

    try:
        module = _load_core_module()
        table = module.read_transition_table_csv(args.transitions_csv, args.p)

        rows_out: list[dict[str, Any]] = []
        with open(args.orderings_csv, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            if "ordering" not in (reader.fieldnames or []):
                raise ValueError("orderings_csv debe contener columna 'ordering'")
            for row in reader:
                ordering_raw = row["ordering"].strip()
                ordering = _parse_ordering(ordering_raw, args.N)
                scores = module.compute_scores(args.p, args.N, ordering, table,
                ↪  return_profiles=False)
                rows_out.append(
                    {
                        "ordering": " ".join(str(x) for x in ordering),
                        "mu_E": scores["mu_E"],
                        "mu_A": scores["mu_A"],
                        "mu_I": scores["mu_I"],
                    }
                )

        out_path = Path(args.output_csv).resolve()
        out_path.parent.mkdir(parents=True, exist_ok=True)
        with out_path.open("w", encoding="utf-8", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=["ordering", "mu_E", "mu_A", "mu_I"])
            writer.writeheader()
            writer.writerows(rows_out)
        print(f"OK: batch guardado en {out_path} ({len(rows_out)} filas)")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgcore-batch-scores: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc
```

5.3 Adaptadores

```
"""Adaptadores de entrada para padic-grn-core."""

from __future__ import annotations

from typing import Any
```

```
def from_grn_table(transitions: dict[int, int], n: int, p: int) -> dict[str, Any]:
    """Adaptador oficial para GRN discretas deterministas.

    Retorna una estructura normalizada para el núcleo de cálculo.
    """
    expected = p**n
    return {
        "kind": "grn_table",
        "N": n,
        "p": p,
        "expected_states": expected,
        "num_transitions": len(transitions),
        "transitions": transitions,
    }


def from_discrete_system(
    transitions: dict[int, int],
    n: int,
    p: int,
    metadata: dict[str, Any] | None = None,
) -> dict[str, Any]:
    """Adaptador experimental para sistemas discretos generales.

    Esta versión beta asume codificación a espacio uniforme p^N.
    """
    payload = from_grn_table(transitions, n, p)
    payload["kind"] = "discrete_system_beta"
    payload["metadata"] = metadata or {}
    return payload
```

5.4 Script de validación

```
"""Valida valores esperados contra evidencia JSON generada."""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple


@dataclass
class CaseSpec:
    name: str
    json_path: Path
    expected_scores: Dict[str, int]
    n: int
    p: int


def load_json(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)
```

```
def check_case(case: CaseSpec) -> Tuple[bool, List[str]]:
    payload = load_json(case.json_path)
    scores = payload["scores"]
    issues: List[str] = []

    # Exact score checks
    for key, expected in case.expected_scores.items():
        got = scores.get(key)
        if got != expected:
            issues.append(f"{key}: esperado={expected}, obtenido={got}")

    # Global identity: mu_E + mu_A + mu_I = (N-1) * p^N
    left = scores["mu_E"] + scores["mu_A"] + scores["mu_I"]
    right = (case.n - 1) * (case.p**case.n)
    if left != right:
        issues.append(f"identidad_global: esperado={right}, obtenido={left}")

    # Per-level identity: E(n)+A(n)+I(n)=p^n
    E = scores.get("E", {})
    A = scores.get("A", {})
    I = scores.get("I", {})
    for level in range(1, case.n):
        k = str(level)
        got = int(E.get(k, 0)) + int(A.get(k, 0)) + int(I.get(k, 0))
        expected = case.p**level
        if got != expected:
            issues.append(f"identidad_nivel_{level}: esperado={expected},
            ↪  obtenido={got}")

    return (len(issues) == 0), issues


def make_report(cases: List[CaseSpec], out_path: Path) -> None:
    lines = [
        "# Validación cruzada automática vs valores esperados",
        "",
        "Fecha: 2026-04-25",
        "",
        "Este reporte compara los JSON de evidencia con valores esperados de artículo y
        ↪  checks matemáticos.",
        "",
    ]

    global_ok = True
    for case in cases:
        ok, issues = check_case(case)
        status = "PASS" if ok else "FAIL"
        lines.append(f"## Caso: {case.name} - {status}")
        lines.append("")
        lines.append(f"- Archivo: `{case.json_path}`")
        lines.append(f"- Esperados: `{case.expected_scores}`")
        if ok:
            lines.append("- Resultado: todos los checks pasaron.")
```

```
        else:
            global_ok = False
            lines.append("- Issues:")
            for issue in issues:
                lines.append(f"  - {issue}")
        lines.append("")

    lines.append(f"## Resultado global: {'PASS' if global_ok else 'FAIL'}")
    lines.append("")
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text("\n".join(lines), encoding="utf-8")


def main() -> None:
    root = Path(__file__).resolve()
    workspace = None
    for p in root.parents:
        if (p / "registro-dossier").exists():
            workspace = p
            break
    if workspace is None:
        raise RuntimeError("No se encontró raíz del workspace.")

    ev = workspace / "registro-dossier" / "06_evidencia_corridas"
    cases = [
        CaseSpec(
            name="Athaliana pi* (N=13, p=2)",
            json_path=ev / "core_scores_athaliana_pi_star.json",
            expected_scores={"mu_E": 26776, "mu_A": 53880, "mu_I": 17648},
            n=13,
            p=2,
        ),
        CaseSpec(
            name="C. elegans pi* (N=8, p=2)",
            json_path=ev / "core_scores_celegans_pi_star.json",
            expected_scores={"mu_E": 956, "mu_A": 392, "mu_I": 444},
            n=8,
            p=2,
        ),
        CaseSpec(
            name="Drosophila pi* (N=6, p=2)",
            json_path=ev / "core_scores_drosophila_pi_star.json",
            expected_scores={"mu_E": 64, "mu_A": 156, "mu_I": 100},
            n=6,
            p=2,
        ),
        CaseSpec(
            name="Toy identidad p=2, N=2",
            json_path=ev / "core_scores_p2.json",
            expected_scores={"mu_E": 0, "mu_A": 0, "mu_I": 4},
            n=2,
            p=2,
        ),
        CaseSpec(
            name="Toy identidad p=3, N=2",
```

```
            json_path=ev / "core_scores_p3.json",
            expected_scores={"mu_E": 0, "mu_A": 0, "mu_I": 9},
            n=2,
            p=3,
        ),
    ]
    out = workspace / "registro-dossier" / "06_evidencia_corridas" /
    ↪  "VALIDACION-CRUZADA-VALORES.md"
    make_report(cases, out)
    print(f"OK: reporte generado en {out}")


if __name__ == "__main__":
    main()
```

6 Pruebas y ejemplos completos

6.1 Prueba smoke

```
from padic_grn_core import __version__


def test_version_defined() -> None:
    assert isinstance(__version__, str)
    assert __version__ == "0.1.0"
```

6.2 Pruebas de adaptadores

```
from padic_grn_core.adapters import from_discrete_system, from_grn_table


def test_from_grn_table_shape() -> None:
    payload = from_grn_table({0: 0, 1: 1, 2: 2, 3: 3}, n=2, p=2)
    assert payload["kind"] == "grn_table"
    assert payload["expected_states"] == 4
    assert payload["num_transitions"] == 4


def test_from_discrete_system_metadata() -> None:
    meta = {"source": "unit-test"}
    payload = from_discrete_system({0: 0, 1: 1, 2: 2, 3: 3}, n=2, p=2, metadata=meta)
    assert payload["kind"] == "discrete_system_beta"
    assert payload["metadata"]["source"] == "unit-test"
```

6.3 Ejemplo de transiciones p=2

```
config_from,config_to
0,0
1,1
2,2
3,3
```

6.4 Ejemplo de transiciones p=3

```
config_from,config_to
0,0
1,1
2,2
3,3
4,4
5,5
6,6
7,7
8,8
```

6.5 Ejemplo de orderings

```
ordering
0 1
1 0
```

7 Evidencia textual de validación

7.1 Log de ejecución de pruebas

```
...                                                                      [100%]
3 passed in 0.01s
```

7.2 Salida JSON toy p=2


```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 2,
  "p": 2,
  "ordering": [
    0,
    1
  ],
  "scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4,
    "p": 2,
    "N": 2,
    "E": {
      "1": 0
    },
    "A": {
      "1": 0
    },
    "I": {
      "1": 2
    }
  }
}
```

7.3 Salida JSON toy p=3


```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 2,
  "p": 3,
  "ordering": [
    0,
    1
  ],
  "scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 9,
    "p": 3,
    "N": 2,
    "E": {
      "1": 0
    },
    "A": {
      "1": 0
    },
    "I": {
      "1": 3
    }
  }
}
```


7.4 Salida CSV batch


```
ordering,mu_E,mu_A,mu_I
0 1,0,0,4
1 0,0,0,4
```


7.5 Salida A. thaliana


```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 13,
  "p": 2,
  "ordering": [
    12,
    3,
    4,
    8,
    5,
    1,
    7,
    2,
    10,
    11,
    9,
    6,
```

```
    0
  ],
  "scores": {
    "mu_E": 26776,
    "mu_A": 53880,
    "mu_I": 17648,
    "p": 2,
    "N": 13,
    "E": {
      "1": 0,
      "2": 4,
      "3": 4,
      "4": 4,
      "5": 8,
      "6": 16,
      "7": 48,
      "8": 46,
      "9": 88,
      "10": 243,
      "11": 112,
      "12": 0
    },
    "A": {
      "1": 0,
      "2": 0,
      "3": 4,
      "4": 12,
      "5": 24,
      "6": 40,
      "7": 70,
      "8": 210,
      "9": 297,
      "10": 781,
      "11": 496,
      "12": 4096
    },
    "I": {
      "1": 2,
      "2": 0,
      "3": 0,
      "4": 0,
      "5": 0,
      "6": 8,
      "7": 10,
      "8": 0,
      "9": 127,
      "10": 0,
      "11": 1440,
      "12": 0
    }
  }
}
```

7.6 Salida C. elegans


```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 8,
  "p": 2,
  "ordering": [
    1,
    0,
    2,
    3,
    6,
    5,
    4,
    7
  ],
  "scores": {
    "mu_E": 956,
    "mu_A": 392,
    "mu_I": 444,
    "p": 2,
    "N": 8,
    "E": {
      "1": 1,
      "2": 3,
      "3": 4,
      "4": 4,
      "5": 20,
      "6": 40,
      "7": 62
    },
    "A": {
      "1": 0,
      "2": 1,
      "3": 2,
      "4": 0,
      "5": 12,
      "6": 9,
      "7": 66
    },
    "I": {
      "1": 1,
      "2": 0,
      "3": 2,
      "4": 12,
      "5": 0,
      "6": 15,
      "7": 0
    }
  }
}
```

7.7 Salida Drosophila


```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 6,
  "p": 2,
  "ordering": [
    0,
    5,
    3,
    4,
    2,
    1
  ],
  "scores": {
    "mu_E": 64,
    "mu_A": 156,
    "mu_I": 100,
    "p": 2,
    "N": 6,
    "E": {
      "1": 0,
      "2": 0,
      "3": 4,
      "4": 6,
      "5": 4
    },
    "A": {
      "1": 0,
      "2": 2,
      "3": 4,
      "4": 9,
      "5": 28
    },
    "I": {
      "1": 2,
      "2": 2,
      "3": 0,
      "4": 1,
      "5": 0
    }
  }
}
```